$$\sum_{j=1}^{n} c_j \bar{x}_j \le \sum_{i=1}^{m} b_i \bar{y}_i \ .$$

Proof We have

$$\begin{aligned}
\sum_{j=1}^{n} c_j \bar{x}_j &\le \sum_{j=1}^{n} \left(\sum_{i=1}^{m} a_{ij} \bar{y}_i \right) \bar{x}_j \quad \text{(by inequalities (29.35))} \\
&= \sum_{i=1}^{m} \left(\sum_{j=1}^{n} a_{ij} \bar{x}_j \right) \bar{y}_i \\
&\le \sum_{i=1}^{m} b_i \bar{y}_i \qquad \text{(by inequalities (29.32))} \ .
\end{aligned}$$

■

Corollary 29.2
Let $\bar{x}$ be a feasible solution to the primal linear program in (29.31)–(29.33), and let $\bar{y}$ be a feasible solution to its dual linear program in (29.34)–(29.36). If

$$\sum_{j=1}^{n} c_j \bar{x}_j = \sum_{i=1}^{m} b_i \bar{y}_i \ ,$$

then $\bar{x}$ and $\bar{y}$ are optimal solutions to the primal and dual linear programs, respectively.

Proof By Lemma 29.1, the objective value of a feasible solution to the primal cannot exceed that of a feasible solution to the dual. The primal linear program is a maximization problem and the dual is a minimization problem. Thus, if feasible solutions $\bar{x}$ and $\bar{y}$ have the same objective value, neither can be improved. ■

We now show that, at optimality, the primal and dual objective values are indeed equal. To prove linear programming duality, we will require one lemma from linear algebra, known as Farkas's lemma, the proof of which Problem 29-4 asks you to provide. Farkas's lemma can take several forms, each of which is about when a set of linear equalities has a solution. In stating the lemma, we use $m + 1$ as a dimension because it matches our use below.

Lemma 29.3 (Farkas's lemma)
Given $M \in \mathbb{R}^{(m+1)\times n}$ and $g \in \mathbb{R}^{m+1}$, exactly one of the following statements is true:

1. There exists $v \in \mathbb{R}^n$ such that $Mv \le g$,
2. There exists $w \in \mathbb{R}^{m+1}$ such that $w \ge 0$, $w^{\mathrm{T}}M = 0$ (an n-vector of all zeros), and $w^{\mathrm{T}}g < 0$.

■

Theorem 29.4 (Linear-programming duality)
Given the primal linear program in (29.31)–(29.33) and its corresponding dual in (29.34)–(29.36), if both are feasible and bounded, then for optimal solutions x^* and y^*, we have $c^{\mathrm{T}}x^* = b^{\mathrm{T}}y^*$.

Proof Let $\mu = b^{\mathrm{T}}y^*$ be the optimal value of the dual linear program given in (29.34)–(29.36). Consider an augmented set of primal constraints in which we add a constraint to (29.31)–(29.33) that the objective value is at least μ. We write out this ***augmented primal*** as

$$Ax \le b\,, \tag{29.47}$$
$$c^{\mathrm{T}}x \ge \mu\,. \tag{29.48}$$

We can multiply (29.48) through by -1 and rewrite (29.47)–(29.48) as

$$\begin{pmatrix} A \\ -c^{\mathrm{T}} \end{pmatrix} x \le \begin{pmatrix} b \\ -\mu \end{pmatrix}. \tag{29.49}$$

Here, $\begin{pmatrix} A \\ -c^{\mathrm{T}} \end{pmatrix}$ denotes an $(m+1)\times n$ matrix, x is an n-vector, and $\begin{pmatrix} b \\ -\mu \end{pmatrix}$ denotes an $(m+1)$-vector.

We claim that if there is a feasible solution $\bar{x}$ to the augmented primal, then the theorem is proved. To establish this claim, observe that $\bar{x}$ is also a feasible solution to the original primal and that it has objective value at least μ. We can then apply Lemma 29.1, which states that the objective value of the primal is at most μ, to complete the proof of the theorem.

It therefore remains to show that the augmented primal has a feasible solution. Suppose, for the purpose of contradiction, that the augmented primal is infeasible, which means that there is no $v \in \mathbb{R}^n$ such that $\begin{pmatrix} A \\ -c^{\mathrm{T}} \end{pmatrix} v \le \begin{pmatrix} b \\ -\mu \end{pmatrix}$. We can apply Farkas's lemma, Lemma 29.3, to inequalty (29.49) with

$$M = \begin{pmatrix} A \\ -c^{\mathrm{T}} \end{pmatrix} \text{ and } g = \begin{pmatrix} b \\ -\mu \end{pmatrix}.$$

Because the augmented primal is infeasible, condition 1 of Farkas's lemma does not hold. Therefore, condition 2 must apply, so that there must exist a $w \in \mathbb{R}^{m+1}$ such that $w \ge 0$, $w^{\mathrm{T}}M = 0$, and $w^{\mathrm{T}}g < 0$. Let's write w as $w = \begin{pmatrix} \bar{y} \\ \lambda \end{pmatrix}$ for some $\bar{y} \in \mathbb{R}^m$ and $\lambda \in \mathbb{R}$, where $\bar{y} \ge 0$ and $\lambda \ge 0$. Substituting for w, M, and g in condition 2 gives

$$\begin{pmatrix} \bar{y} \\ \lambda \end{pmatrix}^{\mathrm{T}} \begin{pmatrix} A \\ -c^{\mathrm{T}} \end{pmatrix} = 0 \text{ and } \begin{pmatrix} \bar{y} \\ \lambda \end{pmatrix}^{\mathrm{T}} \begin{pmatrix} b \\ -\mu \end{pmatrix} < 0\,.$$

Unpacking the matrix notation gives

$$\bar{y}^{\mathrm{T}}A - \lambda c^{\mathrm{T}} = 0 \text{ and } \bar{y}^{\mathrm{T}}b - \lambda\mu < 0 \,. \tag{29.50}$$

We now show that the requirements in (29.50) contradict the assumption that μ is the optimal solution value for the dual linear program. We consider two cases.

The first case is when $\lambda = 0$. In this case, (29.50) simplifies to

$$\bar{y}^{\mathrm{T}}A = 0 \text{ and } \bar{y}^{\mathrm{T}}b < 0 \,. \tag{29.51}$$

We'll now construct a dual feasible solution y' with an objective value smaller than $b^{\mathrm{T}}y^*$. Set $y' = y^* + \epsilon\bar{y}$, for any $\epsilon > 0$. Since

$$\begin{aligned} y'^{\mathrm{T}}A &= (y^* + \epsilon\bar{y})^{\mathrm{T}}A \\ &= y^{*\mathrm{T}}A + \epsilon\bar{y}^{\mathrm{T}}A \\ &= y^{*\mathrm{T}}A && \text{(by (29.51))} \\ &\geq c^{\mathrm{T}} && \text{(because } y^* \text{ is feasible)} \,, \end{aligned}$$

y' is feasible. Now consider the objective value

$$\begin{aligned} b^{\mathrm{T}}y' &= b^{\mathrm{T}}(y^* + \epsilon\bar{y}) \\ &= b^{\mathrm{T}}y^* + \epsilon b^{\mathrm{T}}\bar{y} \\ &< b^{\mathrm{T}}y^* \,, \end{aligned}$$

where the last inequality follows because $\epsilon > 0$ and, by (29.51), $\bar{y}^{\mathrm{T}}b = b^{\mathrm{T}}\bar{y} < 0$ (since both $\bar{y}^{\mathrm{T}}b$ and $b^{\mathrm{T}}\bar{y}$ are the inner product of b and $\bar{y}$), and so their product is negative. Thus we have a feasible dual solution of value less than μ, which contradicts μ being the optimal objective value.

We now consider the second case, where $\lambda > 0$. In this case, we can take (29.50) and divide through by λ to obtain

$$(\bar{y}^{\mathrm{T}}/\lambda)A - (\lambda/\lambda)c^{\mathrm{T}} = 0 \text{ and } (\bar{y}^{\mathrm{T}}/\lambda)b - (\lambda/\lambda)\mu < 0 \,. \tag{29.52}$$

Now set $y' = \bar{y}/\lambda$ in (29.52), giving

$$y'^{\mathrm{T}}A = c^{\mathrm{T}} \text{ and } y'^{\mathrm{T}}b < \mu \,.$$

Thus, y' is a feasible dual solution with objective value strictly less than μ, a contradiction. We conclude that the augmented primal has a feasible solution, and the theorem is proved. ■

Fundamental theorem of linear programming

We conclude this chapter by stating the fundamental theorem of linear programming, which extends Theorem 29.4 to the cases when the linear program may be either feasible or unbounded. Exercise 29.3-8 asks you to provide the proof.

Theorem 29.5 (Fundamental theorem of linear programming)
Any linear program, given in standard form, either

1. has an optimal solution with a finite objective value,
2. is infeasible, or
3. is unbounded. ■

Exercises

29.3-1
Formulate the dual of the linear program given in lines (29.6)–(29.10) on page 852.

29.3-2
You have a linear program that is not in standard form. You could produce the dual by first converting it to standard form, and then taking the dual. It would be more convenient, however, to produce the dual directly. Explain how to directly take the dual of an arbitrary linear program.

29.3-3
Write down the dual of the maximum-flow linear program, as given in lines (29.25)–(29.28) on page 862. Explain how to interpret this formulation as a minimum-cut problem.

29.3-4
Write down the dual of the minimum-cost-flow linear program, as given in lines (29.29)–(29.30) on page 864. Explain how to interpret this problem in terms of graphs and flows.

29.3-5
Show that the dual of the dual of a linear program is the primal linear program.

29.3-6
Which result from Chapter 24 can be interpreted as weak duality for the maximum-flow problem?

29.3-7
Consider the following 1-variable primal linear program:

$$\begin{aligned} &\text{maximize} && tx \\ &\text{subject to} \\ & && rx \le s \\ & && x \ge 0 \ , \end{aligned}$$

where r, s, and t are arbitrary real numbers. State for which values of r, s, and t you can assert that

1. Both the primal linear program and its dual have optimal solutions with finite objective values.
2. The primal is feasible, but the dual is infeasible.
3. The dual is feasible, but the primal is infeasible.
4. Neither the primal nor the dual is feasible.

29.3-8
Prove the fundamental theorem of linear programming, Theorem 29.5.

Problems

29-1 Linear-inequality feasibility
Given a set of m linear inequalities on n variables $x_1, x_2, \ldots, x_n$, the ***linear-inequality feasibility problem*** asks whether there is a setting of the variables that simultaneously satisfies each of the inequalities.

a. Given an algorithm for the linear-programming problem, show how to use it to solve a linear-inequality feasibility problem. The number of variables and constraints that you use in the linear-programming problem should be polynomial in n and m.

b. Given an algorithm for the linear-inequality feasibility problem, show how to use it to solve a linear-programming problem. The number of variables and linear inequalities that you use in the linear-inequality feasibility problem should be polynomial in n and m, the number of variables and constraints in the linear program.

29-2 Complementary slackness
Complementary slackness describes a relationship between the values of primal variables and dual constraints and between the values of dual variables and primal constraints. Let $\bar{x}$ be a feasible solution to the primal linear program given in (29.31)–(29.33), and let $\bar{y}$ be a feasible solution to the dual linear program given in (29.34)–(29.36). Complementary slackness states that the following conditions are necessary and sufficient for $\bar{x}$ and $\bar{y}$ to be optimal:

$$\sum_{i=1}^{m} a_{ij}\bar{y}_i = c_j \text{ or } \bar{x}_j = 0 \text{ for } j = 1, 2, \ldots, n$$

and

$$\sum_{j=1}^{n} a_{ij}\bar{x}_j = b_i \text{ or } \bar{y}_i = 0 \text{ for } i = 1, 2, \ldots, m .$$

a. Verify that complementary slackness holds for the linear program in lines (29.37)–(29.41).

b. Prove that complementary slackness holds for any primal linear program and its corresponding dual.

c. Prove that a feasible solution $\bar{x}$ to a primal linear program given in lines (29.31)–(29.33) is optimal if and only if there exist values $\bar{y} = (\bar{y}_1, \bar{y}_2, \ldots, \bar{y}_m)$ such that

1. $\bar{y}$ is a feasible solution to the dual linear program given in (29.34)–(29.36),
2. $\sum_{i=1}^{m} a_{ij}\bar{y}_i = c_j$ for all j such that $\bar{x}_j > 0$, and
3. $\bar{y}_i = 0$ for all i such that $\sum_{j=1}^{n} a_{ij}\bar{x}_j < b_i$.

29-3 Integer linear programming

An ***integer linear-programming problem*** is a linear-programming problem with the additional constraint that the variables x must take on integer values. Exercise 34.5-3 on page 1098 shows that just determining whether an integer linear program has a feasible solution is NP-hard, which means that there is no known polynomial-time algorithm for this problem.

a. Show that weak duality (Lemma 29.1) holds for an integer linear program.

b. Show that duality (Theorem 29.4) does not always hold for an integer linear program.

c. Given a primal linear program in standard form, let P be the optimal objective value for the primal linear program, D be the optimal objective value for its dual, IP be the optimal objective value for the integer version of the primal (that is, the primal with the added constraint that the variables take on integer values), and ID be the optimal objective value for the integer version of the dual. Assuming that both the primal integer program and the dual integer program are feasible and bounded, show that

$$IP \le P = D \le ID .$$

29-4 Farkas's lemma

Prove Farkas's lemma, Lemma 29.3.

29-5 Minimum-cost circulation

This problem considers a variant of the minimum-cost-flow problem from Section 29.2 in which there is no demand, source, or sink. Instead, the input, as before, contains a flow network, capacity constraints $c(u, v)$, and edge costs $a(u, v)$. A flow is feasible if it satisfies the capacity constraint on every edge and flow conservation at *every* vertex. The goal is to find, among all feasible flows, the one of minimum cost. We call this problem the ***minimum-cost-circulation problem.***

a. Formulate the minimum-cost-circulation problem as a linear program.

b. Suppose that for all edges $(u, v) \in E$, we have $a(u, v) > 0$. What does an optimal solution to the minimum-cost-circulation problem look like?

c. Formulate the maximum-flow problem as a minimum-cost-circulation problem linear program. That is, given a maximum-flow problem instance $G = (V, E)$ with source s, sink t and edge capacities c, create a minimum-cost-circulation problem by giving a (possibly different) network $G' = (V', E')$ with edge capacities c' and edge costs a' such that you can derive a solution to the maximum-flow problem from a solution to the minimum-cost-circulation problem.

d. Formulate the single-source shortest-path problem as a minimum-cost-circulation problem linear program.

Chapter notes

This chapter only begins to study the wide field of linear programming. A number of books are devoted exclusively to linear programming, including those by Chvátal [94], Gass [178], Karloff [246], Schrijver [398], and Vanderbei [444]. Many other books give a good coverage of linear programming, including those by Papadimitriou and Steiglitz [353] and Ahuja, Magnanti, and Orlin [7]. The coverage in this chapter draws on the approach taken by Chvátal.

The simplex algorithm for linear programming was invented by G. Dantzig in 1947. Shortly after, researchers discovered how to formulate a number of problems in a variety of fields as linear programs and solve them with the simplex algorithm. As a result, applications of linear programming flourished, along with several algorithms. Variants of the simplex algorithm remain the most popular

methods for solving linear-programming problems. This history appears in a number of places, including the notes in [94] and [246].

The ellipsoid algorithm was the first polynomial-time algorithm for linear programming and is due to L. G. Khachian in 1979. It was based on earlier work by N. Z. Shor, D. B. Judin, and A. S. Nemirovskii. Grötschel, Lovász, and Schrijver [201] describe how to use the ellipsoid algorithm to solve a variety of problems in combinatorial optimization. To date, the ellipsoid algorithm does not appear to be competitive with the simplex algorithm in practice.

Karmarkar's paper [247] includes a description of the first interior-point algorithm. Many subsequent researchers designed interior-point algorithms. Good surveys appear in the article of Goldfarb and Todd [189] and the book by Ye [463].

Analysis of the simplex algorithm remains an active area of research. V. Klee and G. J. Minty constructed an example on which the simplex algorithm runs through $2^n - 1$ iterations. The simplex algorithm usually performs well in practice, and many researchers have tried to give theoretical justification for this empirical observation. A line of research begun by K. H. Borgwardt, and carried on by many others, shows that under certain probabilistic assumptions on the input, the simplex algorithm converges in expected polynomial time. Spielman and Teng [421] made progress in this area, introducing the "smoothed analysis of algorithms" and applying it to the simplex algorithm.

The simplex algorithm is known to run efficiently in certain special cases. Particularly noteworthy is the network-simplex algorithm, which is the simplex algorithm, specialized to network-flow problems. For certain network problems, including the shortest-paths, maximum-flow, and minimum-cost-flow problems, variants of the network-simplex algorithm run in polynomial time. See, for example, the article by Orlin [349] and the citations therein.

30 Polynomials and the FFT

The straightforward method of adding two polynomials of degree n takes $\Theta(n)$ time, but the straightforward method of multiplying them takes $\Theta(n^2)$ time. This chapter will show how the fast Fourier transform, or FFT, can reduce the time to multiply polynomials to $\Theta(n \lg n)$.

The most common use for Fourier transforms, and hence the FFT, is in signal processing. A signal is given in the ***time domain***: as a function mapping time to amplitude. Fourier analysis expresses the signal as a weighted sum of phase-shifted sinusoids of varying frequencies. The weights and phases associated with the frequencies characterize the signal in the ***frequency domain***. Among the many everyday applications of FFT's are compression techniques used to encode digital video and audio information, including MP3 files. Many fine books delve into the rich area of signal processing, and the chapter notes reference a few of them.

Polynomials

A ***polynomial*** in the variable x over an algebraic field F represents a function $A(x)$ as a formal sum:

$$A(x) = \sum_{j=0}^{n-1} a_j x^j \ .$$

The values $a_0, a_1, \ldots, a_{n-1}$ are the ***coefficients*** of the polynomial. The coefficients and x are drawn from a field F, typically the set $\mathbb{C}$ of complex numbers. A polynomial $A(x)$ has ***degree*** k if its highest nonzero coefficient is a_k, in which case we say that $\text{degree}(A) = k$. Any integer strictly greater than the degree of a polynomial is a ***degree-bound*** of that polynomial. Therefore, the degree of a polynomial of degree-bound n may be any integer between 0 and $n - 1$, inclusive.

A variety of operations extend to polynomials. For ***polynomial addition***, if $A(x)$ and $B(x)$ are polynomials of degree-bound n, their ***sum*** is a polynomial $C(x)$, also

of degree-bound n, such that $C(x) = A(x) + B(x)$ for all x in the underlying field. That is, if

$$A(x) = \sum_{j=0}^{n-1} a_j x^j \quad \text{and} \quad B(x) = \sum_{j=0}^{n-1} b_j x^j ,$$

then

$$C(x) = \sum_{j=0}^{n-1} c_j x^j ,$$

where $c_j = a_j + b_j$ for $j = 0, 1, \ldots, n-1$. For example, given the polynomials $A(x) = 6x^3 + 7x^2 - 10x + 9$ and $B(x) = -2x^3 + 4x - 5$, their sum is $C(x) = 4x^3 + 7x^2 - 6x + 4$.

For ***polynomial multiplication***, if $A(x)$ and $B(x)$ are polynomials of degree-bound n, their ***product*** $C(x)$ is a polynomial of degree-bound $2n-1$ such that $C(x) = A(x)B(x)$ for all x in the underlying field. You probably have multiplied polynomials before, by multiplying each term in $A(x)$ by each term in $B(x)$ and then combining terms with equal powers. For example, you can multiply $A(x) = 6x^3 + 7x^2 - 10x + 9$ and $B(x) = -2x^3 + 4x - 5$ as follows:

$$\begin{array}{rrrrrrr}
 & & & 6x^3 & + 7x^2 & - 10x & + 9 \\
 & & & - 2x^3 & & + 4x & - 5 \\
\hline
 & & & - 30x^3 & - 35x^2 & + 50x & - 45 \quad \text{(multiply } A(x) \text{ by } -5) \\
 & & 24x^4 & + 28x^3 & - 40x^2 & + 36x & \quad \text{(multiply } A(x) \text{ by } 4x) \\
- 12x^6 & - 14x^5 & + 20x^4 & - 18x^3 & & & \quad \text{(multiply } A(x) \text{ by } -2x^3) \\
\hline
- 12x^6 & - 14x^5 & + 44x^4 & - 20x^3 & - 75x^2 & + 86x & - 45
\end{array}$$

Another way to express the product $C(x)$ is

$$C(x) = \sum_{j=0}^{2n-2} c_j x^j , \tag{30.1}$$

where

$$c_j = \sum_{k=0}^{j} a_k b_{j-k} , \tag{30.2}$$

(By the definition of degree, $a_k = 0$ for all $k >$ degree(A) and $b_k = 0$ for all $k >$ degree(B).) If A is a polynomial of degree-bound n_a and B is a polynomial of degree-bound n_b, then C must be a polynomial of degree-bound $n_a + n_b - 1$, because degree(C) = degree(A) + degree(B). Since a polynomial of degree-bound k is also a polynomial of degree-bound $k+1$, we normally make the somewhat simpler statement that the product polynomial C is a polynomial of degree-bound $n_a + n_b$.